

Django Vue Lab

Занятие 2. Backend разработка, MVC

Черепанов И.Н. Филиппев В.А.

Backend серверная разработка



Backend

- web-сервер
- Бизнес логика
- Базы данных

Основные подходы

- Статические страницы (HTML – без программирования)
- Динамические страницы – требуют программирования
 - Писать с нуля - лучше не делать
 - **Framework**
 - Django
 - Flask
 - FastApi
 - Ruby on Rails
 - Laravel
 - Express
 - CMS
 - Wordpress
 - Bitrix
 - DjangnoCMS
 - ...

На чем писать?

- PHP
- Perl
- C#
- **Python**
- Java
- JavaScript
- **Go**
- **Rust**
- и прочие...

Способы реализации серверной части

- Внедрить код в web-сервер
- Использовать модули
- CGI
- fastCGI
- **WSGI**
- **ASGI**

CGI

```
import os
import sys

print("Content-type: text/html")
print("Status: 200")
print("")
print("<h1>Hello world!</h1>")
```

Пример Запрос - Ответ

```
GET /cgi-bin/index.py HTTP/1.1
> Host: localhost:8001
> User-Agent: curl/8.20.0
> Accept: */*
>
* Request completely sent off
* HTTP 1.0, assume close after body
< HTTP/1.0 200 Script output follows
< Server: SimpleHTTP/0.6 Python/3.14.5
< Date: Fri, 22 May 2026 13:00:43 GMT
< Content-Type: text/html; charset=utf-8
<
<h1>Hello world!</h1>
```

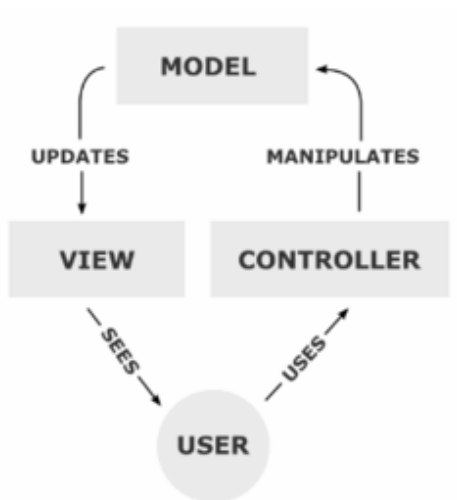

MVC

Model-View-Controller (**MVC**, «Модель-Представление-Контроллер») — схема разделения данных приложения, пользовательского интерфейса и управляющей логики на три отдельных компонента: модель, представление и контроллер — таким образом, что модификация каждого компонента может осуществляться независимо.

- **Модель** (Model) предоставляет данные и реагирует на команды контроллера, изменяя свое состояние
- **Представление** (View) отвечает за отображение данных модели пользователю, реагируя на изменения модели
- **Контроллер** (Controller) интерпретирует действия пользователя, оповещая модель о необходимости изменений

MVC

- **Model** – работа с данными (БД)
- **Controller** – логика работы
- **View** – как выглядит
- Роутинг – соответствие запроса контроллеру



Django

Django (Джанго) — свободный фреймворк для веб-приложений на языке Python, использующий шаблон проектирования MVC. Проект поддерживается организацией Django Software Foundation.

- Model – Model
- Controller – View
- View – Template

Django

```
from django.db import models

class Article(models.Model):
    name = models.CharField(
        max_length=255,
        verbose_name='название',
    )
    discript = models.CharField(
        verbose_name='описание',
        max_length=255,
    )
```

ORM

SQL – не нужен?

ORM (англ. Object-Relational Mapping, объектно-реляционное отображение)

```
./manage.py makemigrations  
./manage.py migrate
```

```
art_list = Article.objects.filter(name='Test')  
art = Article(name='Test', description='test')  
art.save()
```

```
art = Article()  
art.name = 'test'  
art.description = 'test'  
art.save()
```

View

View - функция принимает HttpRequest, Возвращает HttpResponse

```
def current_datetime(request):  
    now = datetime.datetime.now()  
    html = "<html><body>It is now %s.</body></html>" % now  
    return HttpResponse(html)
```

View

```
from django.shortcuts import render

def detail(request, article_id):
    art = Article.objects.get(pk=poll_id)
    return render(request, 'article/detail.html', {'article':
art})
```

Классовые view

```
class ArticleDetailView(DetailView):
    model = Article
```

Routing

Какая функция отработает на каком URL? **urls.py**

```
from django.conf.urls import url
from . import views

urlpatterns = [
    url(r'^articles/2003/$', views.special_case_2003),
    url(r'^articles/([0-9]{4})/$', views.year_archive),
    url(r'^articles/([0-9]{4})/([0-9]{2})/$',
        views.month_archive),
]
```


Routing

```
from django.urls import path

def detail(request, question_id):
    return HttpResponse("You're looking at question %s." %
        question_id)

def vote(request, question_id):
    return HttpResponse("You're voting on question %s." %
        question_id)

urlpatterns = [
    path("", views.index, name="index"),
    # ex: /polls/5/
    path("<int:question_id>/", views.detail, name="detail"),
    # ex: /polls/5/results/
    path("<int:question_id>/vote/", views.vote, name="vote"),
]
```

Шаблоны

Шаблоны содержат статический HTML и динамические данные, рендеринг которых описан специальным синтаксисом.

```
<h1>{{ question.question_text }}</h1>
<ul>
{% for choice in question.choice_set.all %}
<li>{{ choice.choice_text }}</li>
{% endfor %}
</ul>
```

```
def detail(request, question_id):
    question = get_object_or_404(Question, pk=question_id)
    return render(request, 'polls/detail.html', {'question':
question})
```

Шаблоны

- язык шаблонов похож на jinja2
- шаблоны могут наследоваться
- избегайте `hadr-code url`
- Можно вызывать функции без параметров
- tags
- filters
- request
- После отрисовки передается пользователю
- нет связи с сервером.

Спасибо за внимание